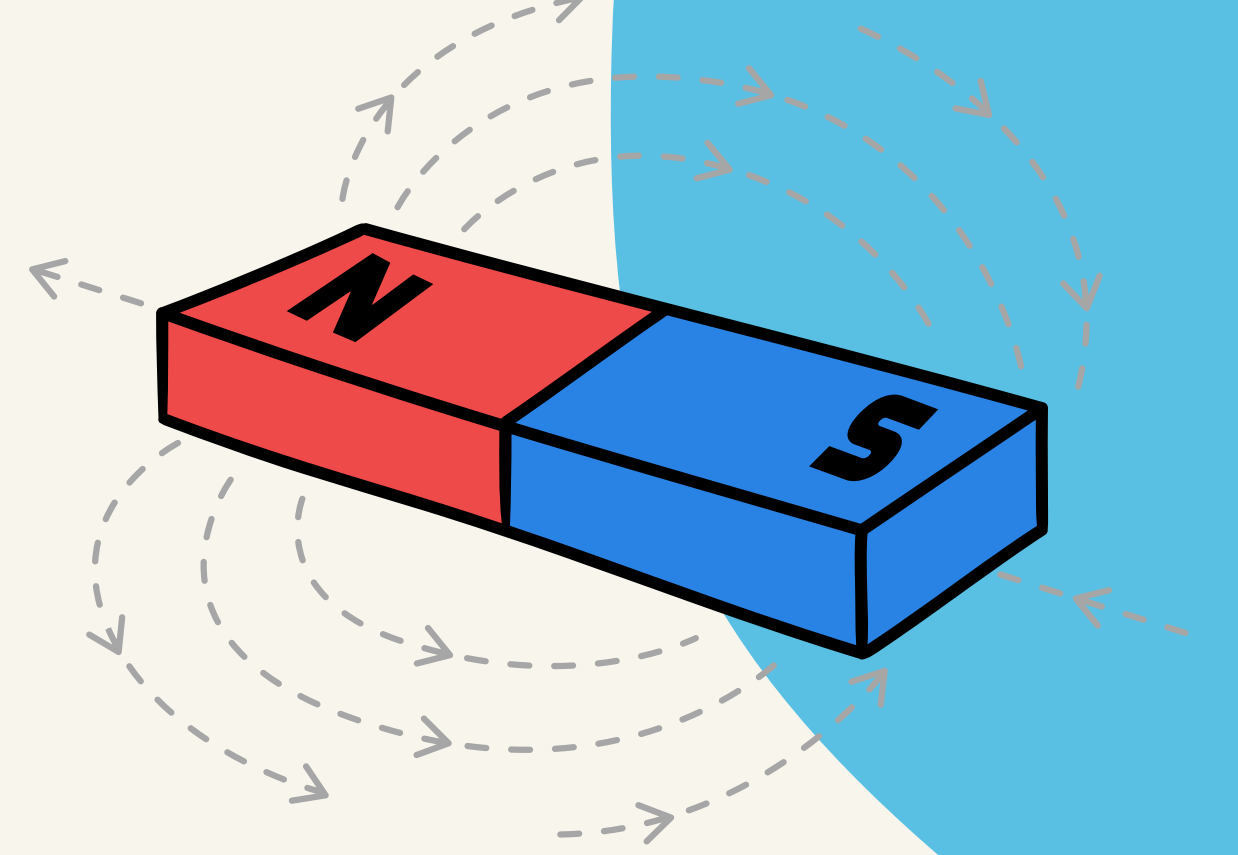
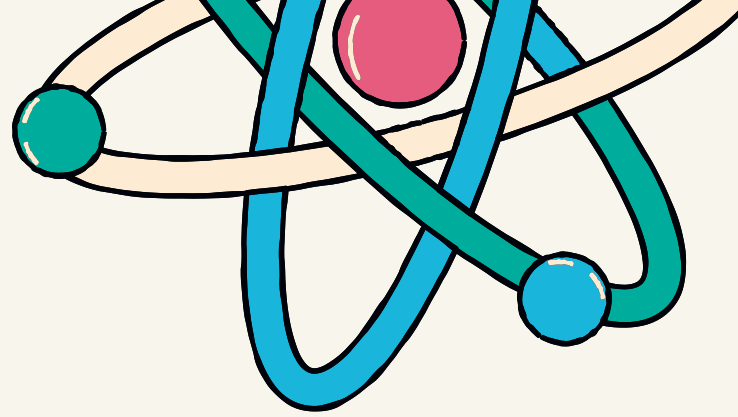




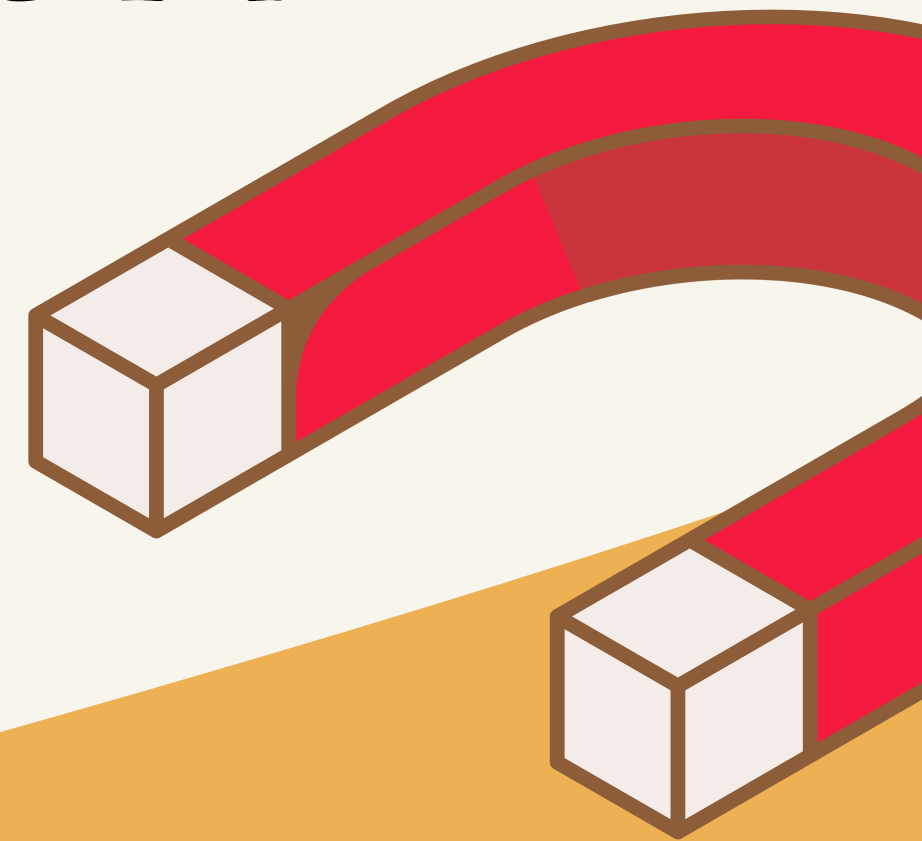
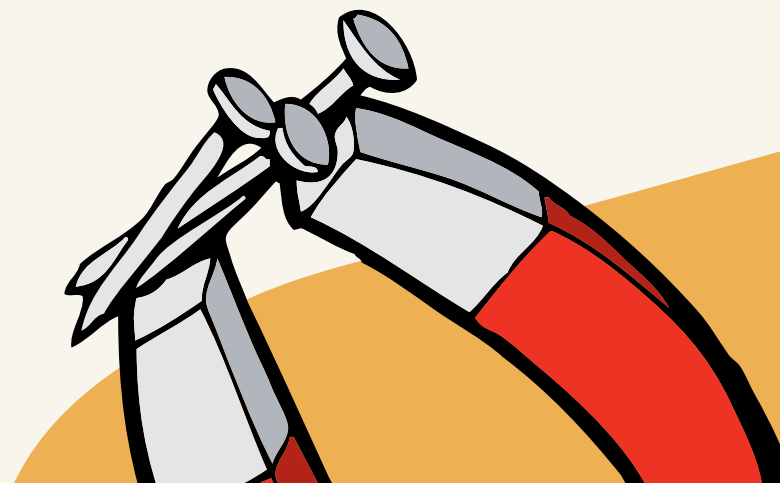
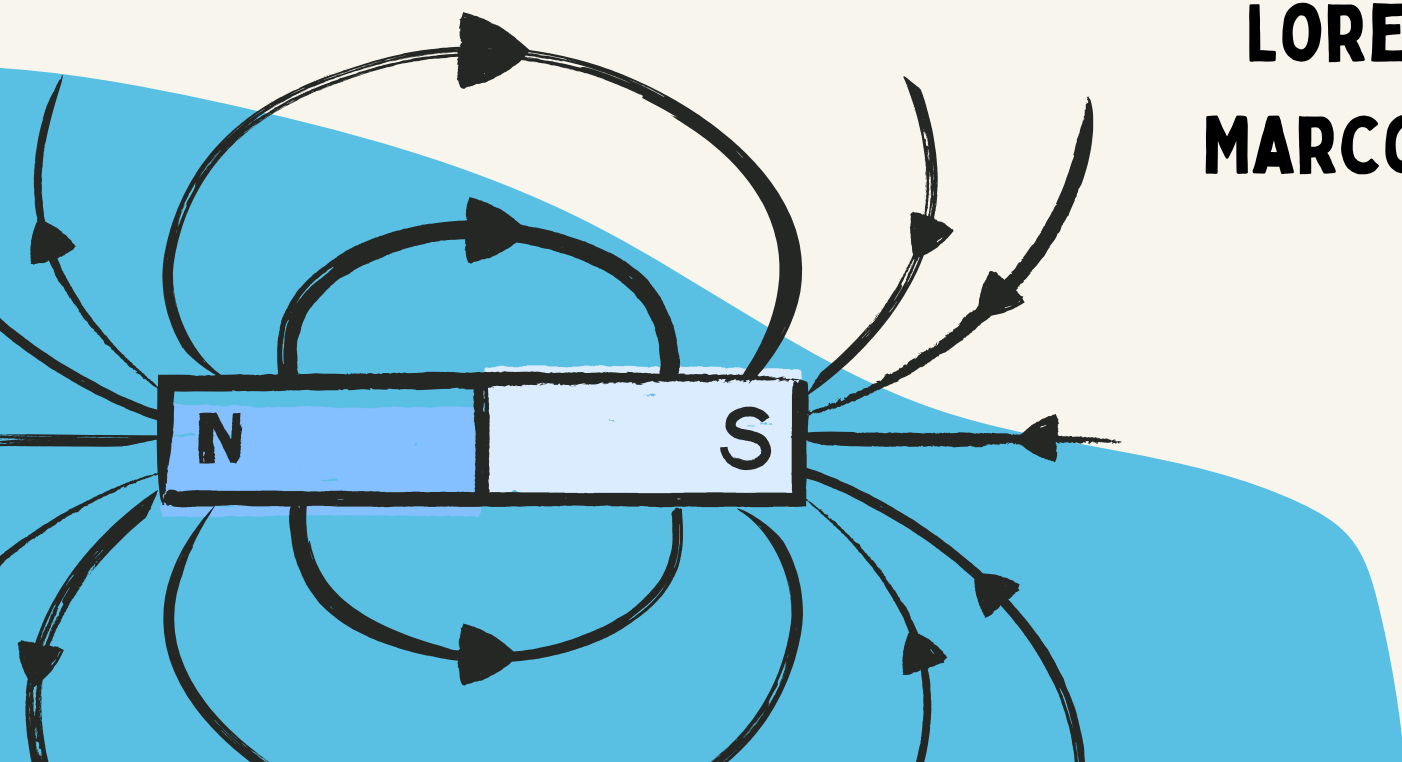
ELECTROMAGNETISM

DANIEL HERNÁNDEZ GUTIÉRREZ A01640706

CARLOS GERARDO GUYONNET SÁNCHEZ A01647279

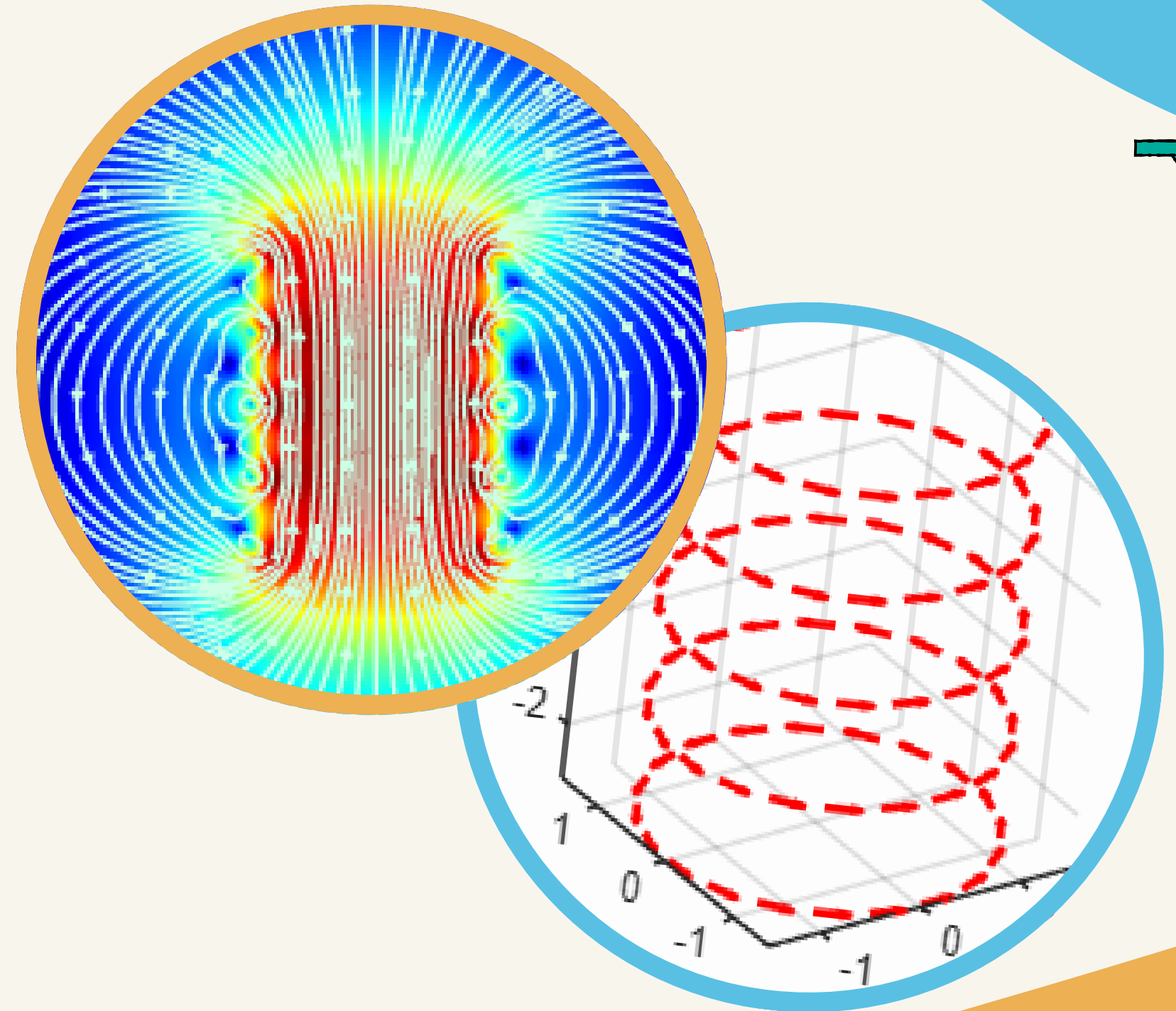
LORENZO ORRANTE ROMÁN A01641580

MARCO NATSUMI RABIELA MUN A01647377

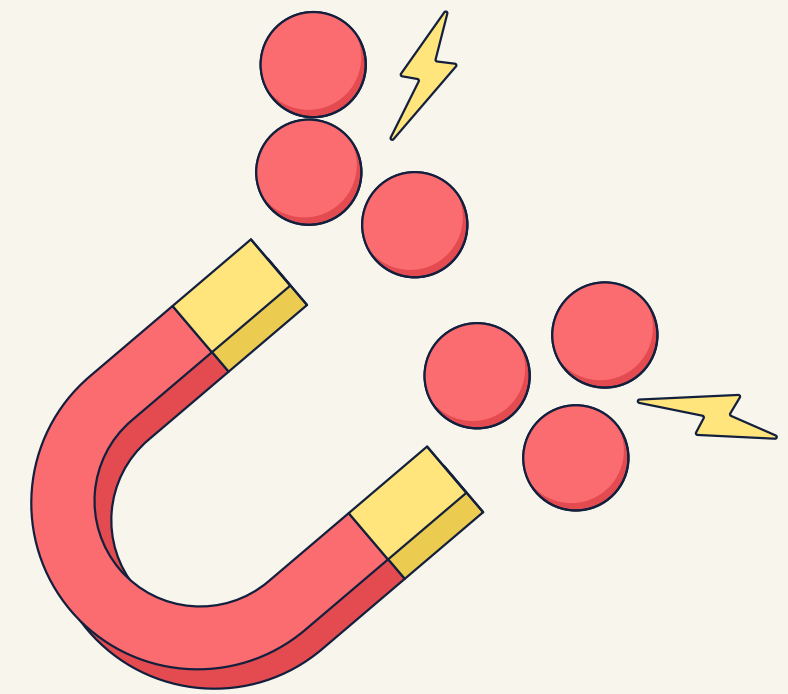


INTRODUCTION

The main objective of the experiment with the coils is to visualize the magnetic field generated by a solenoid, identifying the regions where the magnetic field has the greatest intensity.



THEORETICAL FRAMEWORK



01

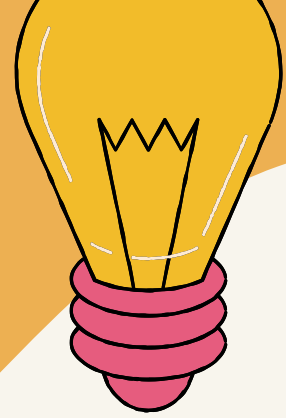
MAGNETISM

Magnetism is a manifestation of one of the fundamental forces of nature: the electromagnetic force. It arises from the movement of electric charges, especially electrons within atoms, and is expressed through forces between magnetic materials or electric currents. In other words, it is a physical phenomenon by which objects exert attractive or repulsive forces on other materials.

02

MAGNETIC FIELD

The magnetic field is a region of space where a moving charge (such as an electric current) experiences a magnetic force. It is a vector field, meaning that at every point in space it has both a direction and a magnitude.



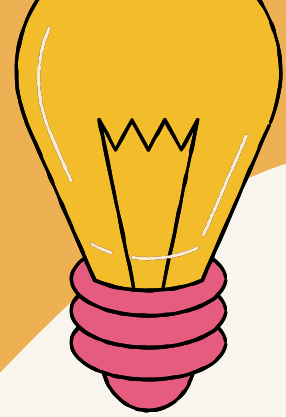
BIOT-SAVART LAW

The Biot-Savart law describes the magnetic field generated by a steady electric current. The direction, magnitude, and orientation of the field depend on:

- The direction and magnitude of the current.
- The shape of the conductor.
- The distance between the observation point and the current element.



$$\vec{B} = \frac{\mu_0 I}{4\pi} \int \frac{\vec{dl} \times \vec{r}}{r^3}$$



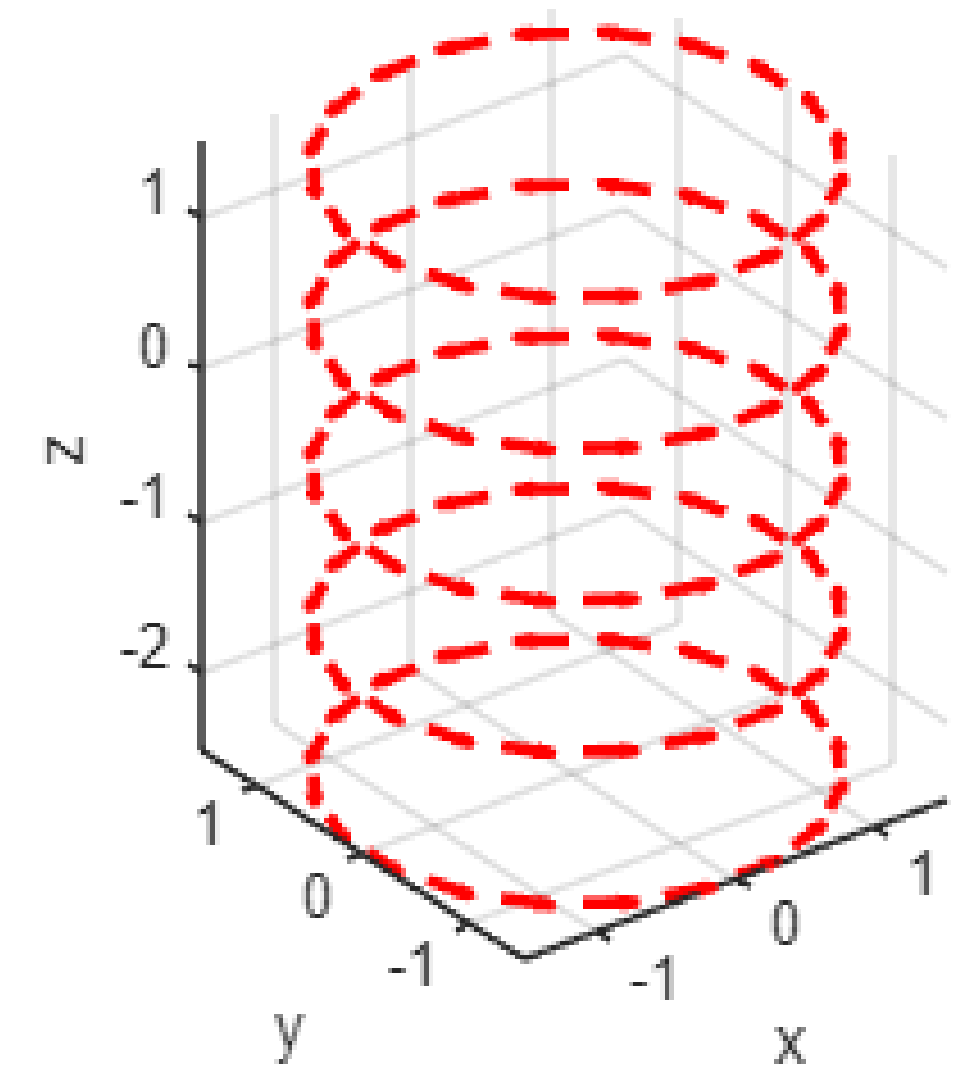
VARIABLES INITIALIZATION AND COIL PLOTTING

At the beginning of the code, we defined the points for each coil, the derivative of theta and its angle, the spacing between coils, and unit values for calculating the magnetic field, among other parameters. A for loop is used to plot each point of the coil and the total number of coils, which are visualized using quiver to display the direction of the magnetic field with arrows.

```
N = 20;  
s = 1;  
ne = 5;  
dtheta = (2*pi)/N;  
angulo = 0:dtheta:(2*pi - dtheta);  
radio = 1.5;  
h = 1;  
I = 300;  
mo = 4*pi*1e-7;  
km = mo * I / (4*pi);  
rw = 0.2;
```

```
for i = 1:ne  
    x(s:s + N - 1) = radio * cos(angulo);  
    y(s:s + N - 1) = radio * sin(angulo);  
    z(s:s + N - 1) = -ne/2 * h + (i - 1) * h;  
    dx(s:s + N - 1) = -y(s:s + N - 1) * dtheta;  
    dy(s:s + N - 1) = x(s:s + N - 1) * dtheta;  
    s = s + N;  
end
```

Corriente en espiras



MAGNETIC FIELD CALCULATION

After plotting the solenoid, we defined the area for the magnetic field visualization and created zero vectors to store the components of the magnetic field.

FOR LOOP

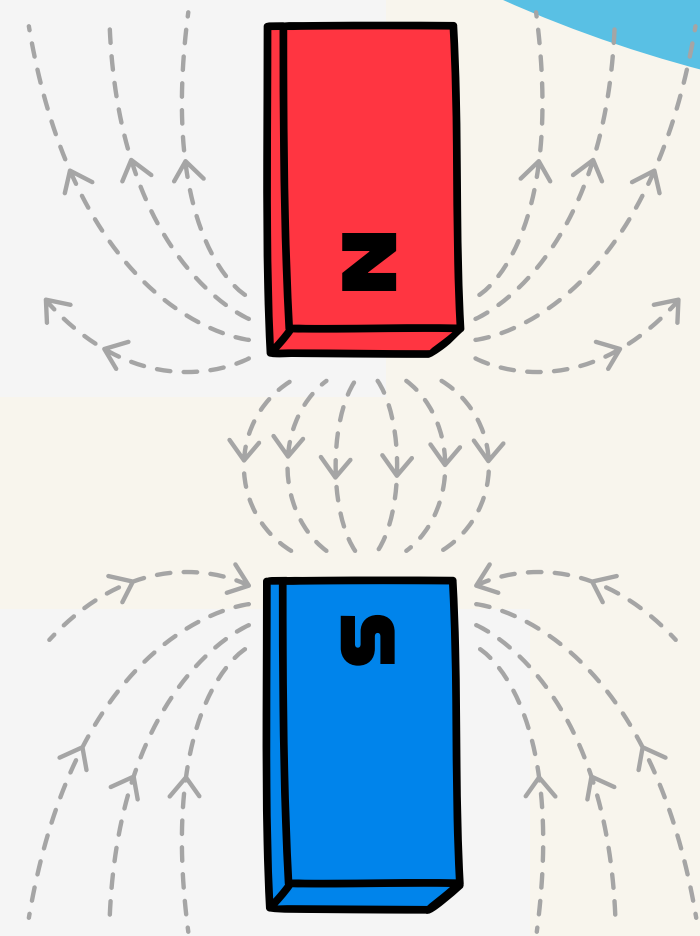
We then proceeded to calculate the magnetic field in order to plot it. This was done using a triple for loop, which first computes the distances of each component between the evaluation points. Then, using the Biot-Savart law, the components of the magnetic field at each point are calculated.

```
xcampo = -5:0.1:5; ycampo = xcampo; zcampo = xcampo;
```

```
lx = length(xcampo);  
ly = length(ycampo);  
lz = length(zcampo);
```

```
Bx = zeros(lx, ly, lz);  
By = zeros(lx, ly, lz);  
Bz = zeros(lx, ly, lz);
```

```
tic  
for i = 1:lx  
    for j = 1:ly  
        for k = 1:lz  
            for l = 1:ne*N  
                rx = xcampo(i) - x(l);  
                ry = ycampo(j) - y(l);  
                rz = zcampo(k) - z(l);  
  
                r = sqrt(rx^2 + ry^2 + rz^2 + rw^2);  
                r3 = r^3;  
  
                Bx(i, j, k) = Bx(i, j, k) + km * (dy(l) * rz - dz(l) * ry) / r3;  
                By(i, j, k) = By(i, j, k) + km * (dz(l) * rx - dx(l) * rz) / r3;  
                Bz(i, j, k) = Bz(i, j, k) + km * (dx(l) * ry - dy(l) * rx) / r3;  
            end  
        end  
    end  
end  
toc
```



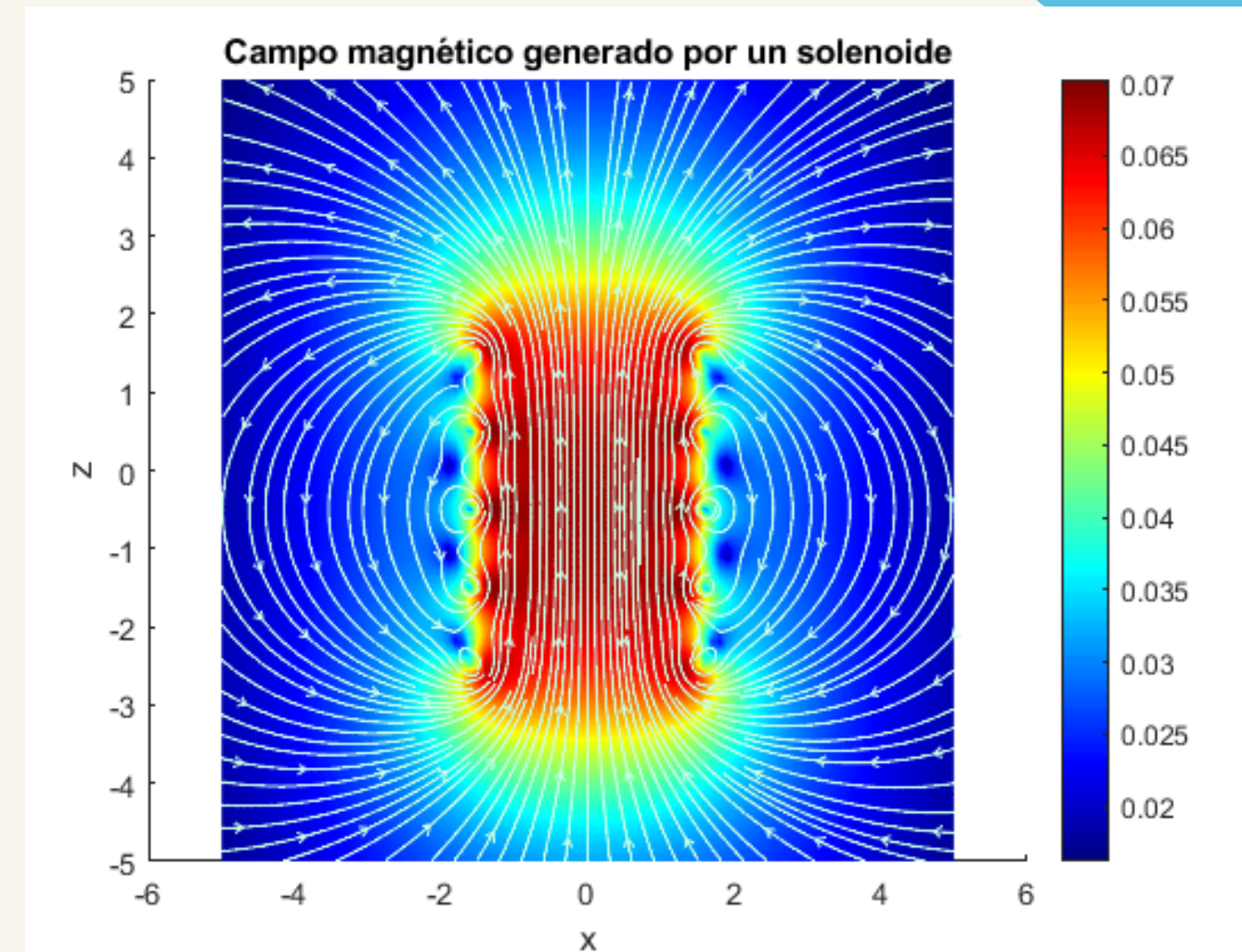
MAGNETIC FIELD GRAPHIC

```
% Graficar plano XZ
Bmag = sqrt(Bx.^2 + By.^2 + Bz.^2);
centery = round(ly / 2);
Bx_xz = squeeze(Bx(:, centery, :));
Bz_xz = squeeze(Bz(:, centery, :));
Bxz = squeeze(Bmag(:, centery, :));

figure(2);
hold on;
pcolor(xcampo, zcampo, (Bxz').^(1/3)); shading interp; colormap jet; colorbar;
h1 = streamslice(xcampo, zcampo, Bx_xz', Bz_xz', 3);
set(h1, 'Color', [0.8 1.0 0.9]);

xlabel('x');
ylabel('z');
title('Campo magnético generado por un solenoide');
```

The calculation begins with the magnitude of the magnetic field, as it will be used to visually represent the field in 2D using centery and squeeze..



To identify the intensity of the magnetic field, we use pcolor to generate a color map, allowing the magnitude at each part of the graph to be visually distinguished through color.

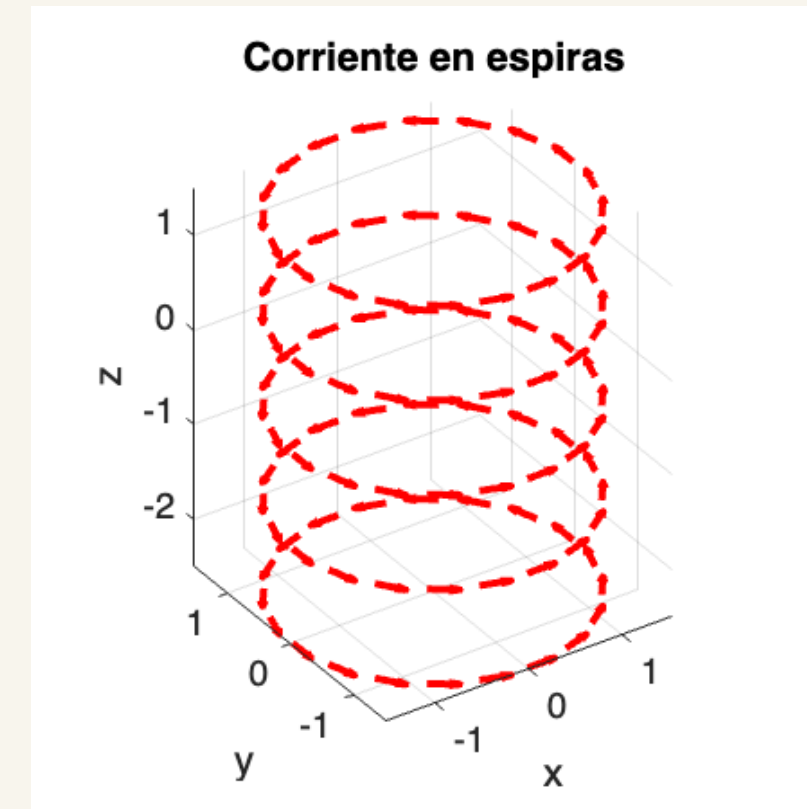
COIL FUNCTION

```
function [Px, Py, Pz, dx, dy, dz] = espiras(ne, N, radio)
    dtheta = (2*pi)/N;
    angulo = 0:dtheta:(2*pi - dtheta);
    s = 1;
    h = 1;
    for i = 1:ne
        Px(s:s + N - 1) = radio * cos(angulo);
        Py(s:s + N - 1) = radio * sin(angulo);
        Pz(s:s + N - 1) = -ne/2 * h + (i - 1) * h;
        dx(s:s + N - 1) = -Py(s:s + N - 1) * dtheta;
        dy(s:s + N - 1) = Px(s:s + N - 1) * dtheta;
        s = s + N;
    end

    dz = zeros(1, N * ne);

    figure(1);
    quiver3(Px, Py, Pz, dx, dy, dz, 0.5, '-r', 'LineWidth', 2);
    view(-34, 33);
    title('Corriente en espiras');
    xlabel('x'); ylabel('y'); zlabel('z');
    axis equal;
    grid on;
end
```

The coil function generates a set of circular loops, modeling the position and direction of the wire. It simulates a solenoid composed of multiple circular turns through which current flows.



The input parameters are ne (number of coils), N (discrete points per coil), and the radius of each coil.

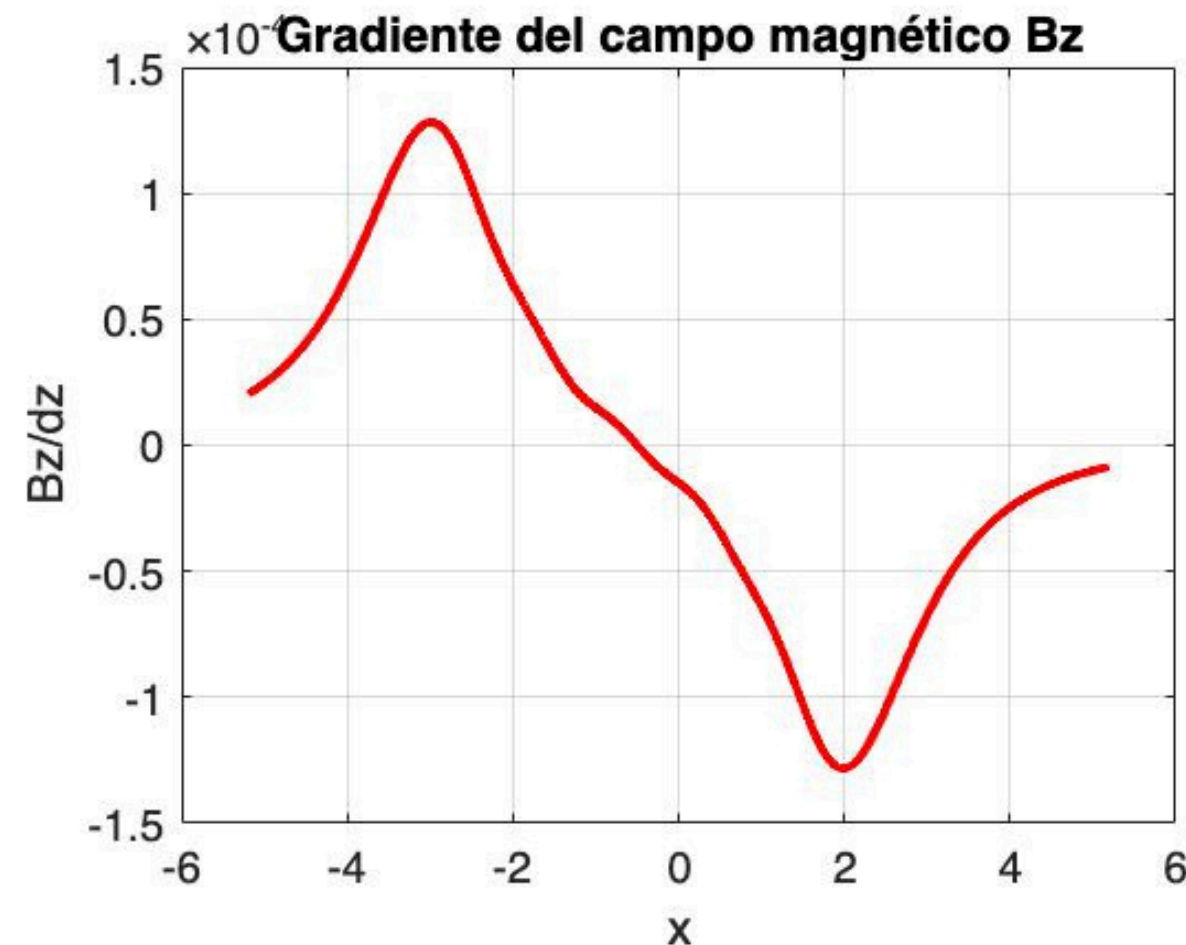
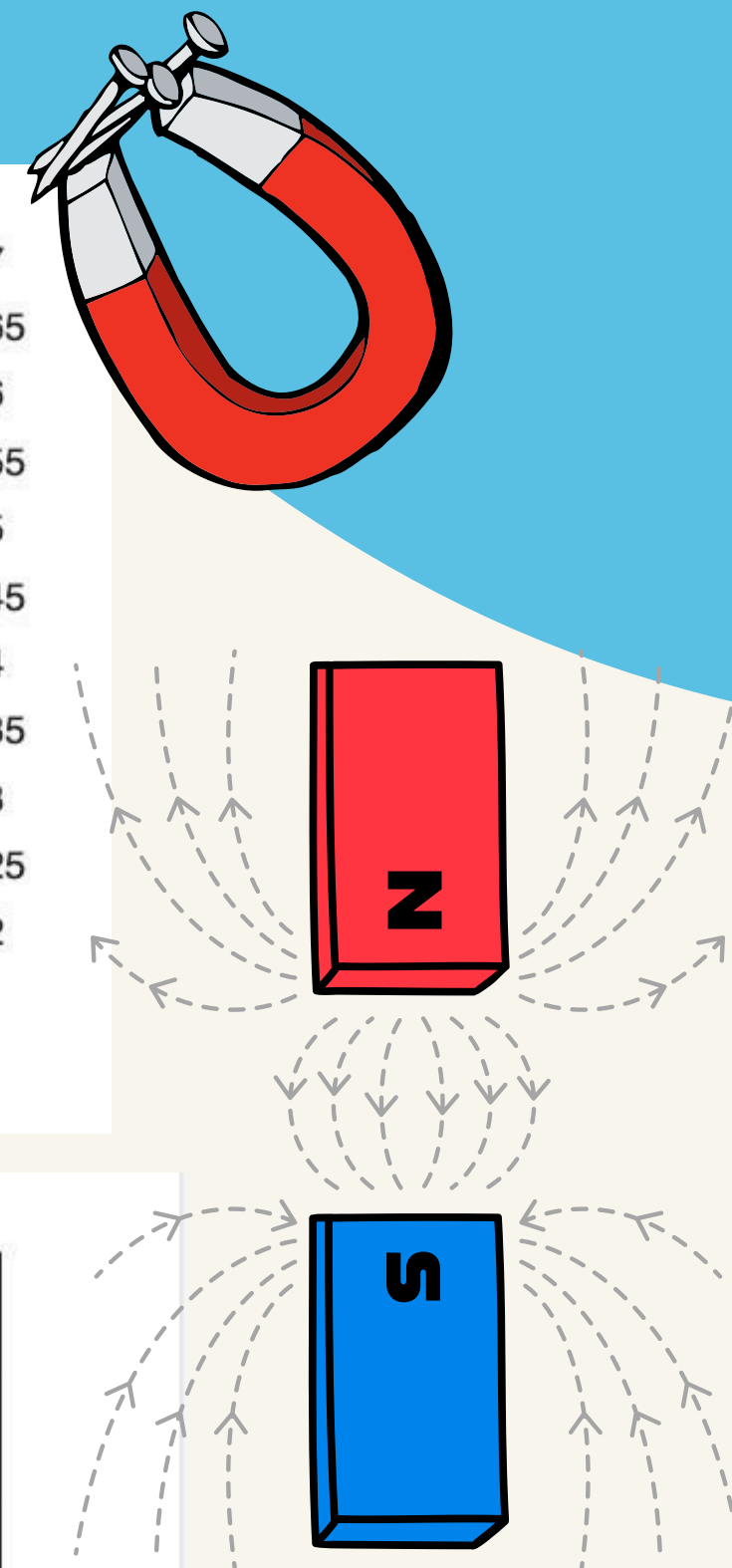
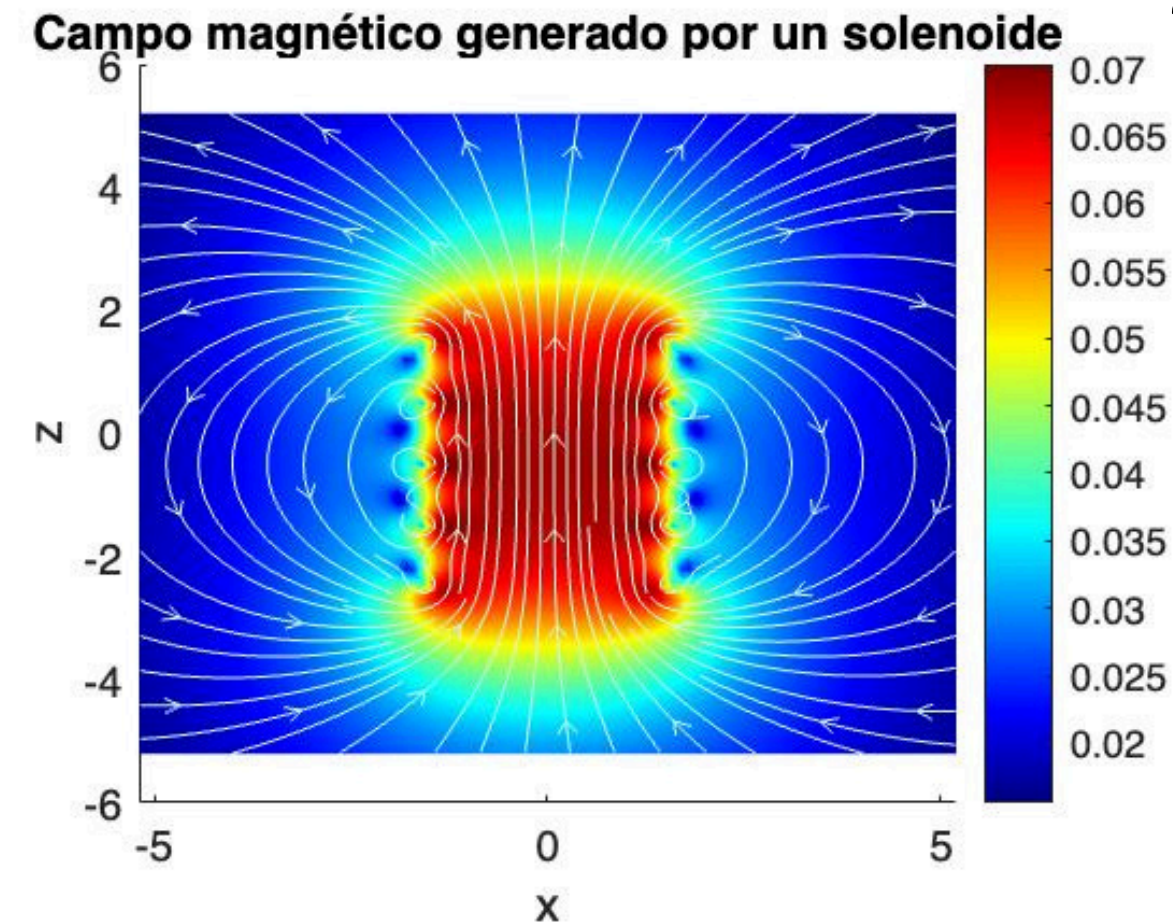
The output parameters are Px, Py, and Pz (3D coordinates of the points along the coil) and dx, dy, and dz (direction of the current at each point).

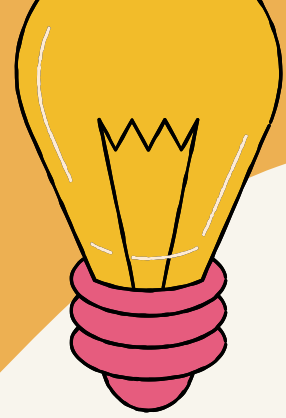
CAMPO_B FUNCTION

The campoB function calculates the magnetic field of a solenoid using the Biot-Savart law and, depending on the selected option, generates either a field map in the x-z plane or the axial gradient profile dBz/dz. These visualizations allow for analysis of how the magnetic field varies in space.

PARAMETERS

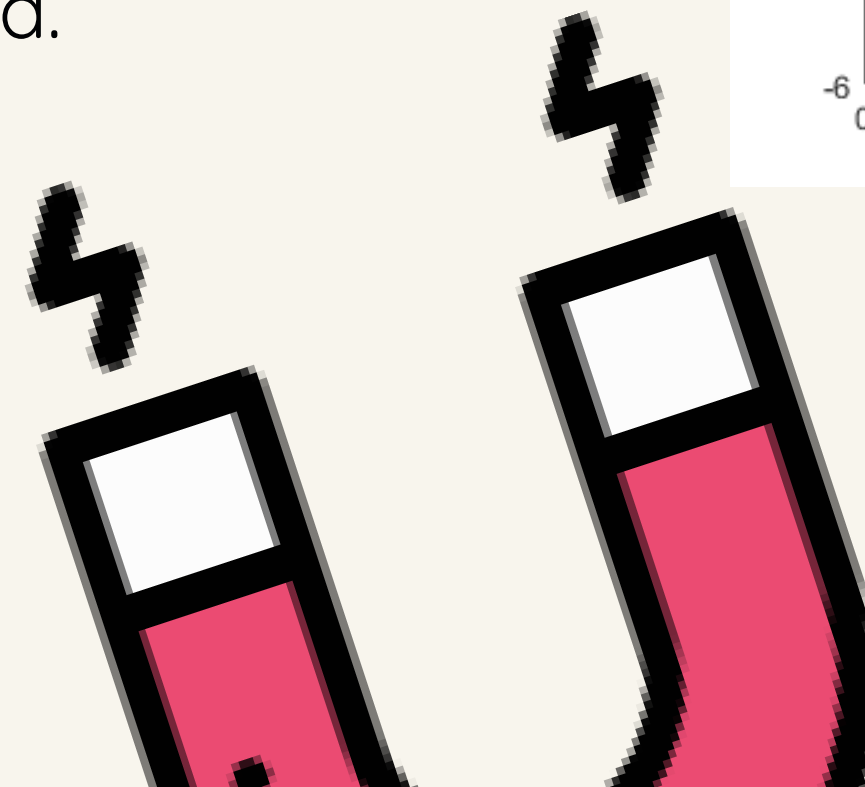
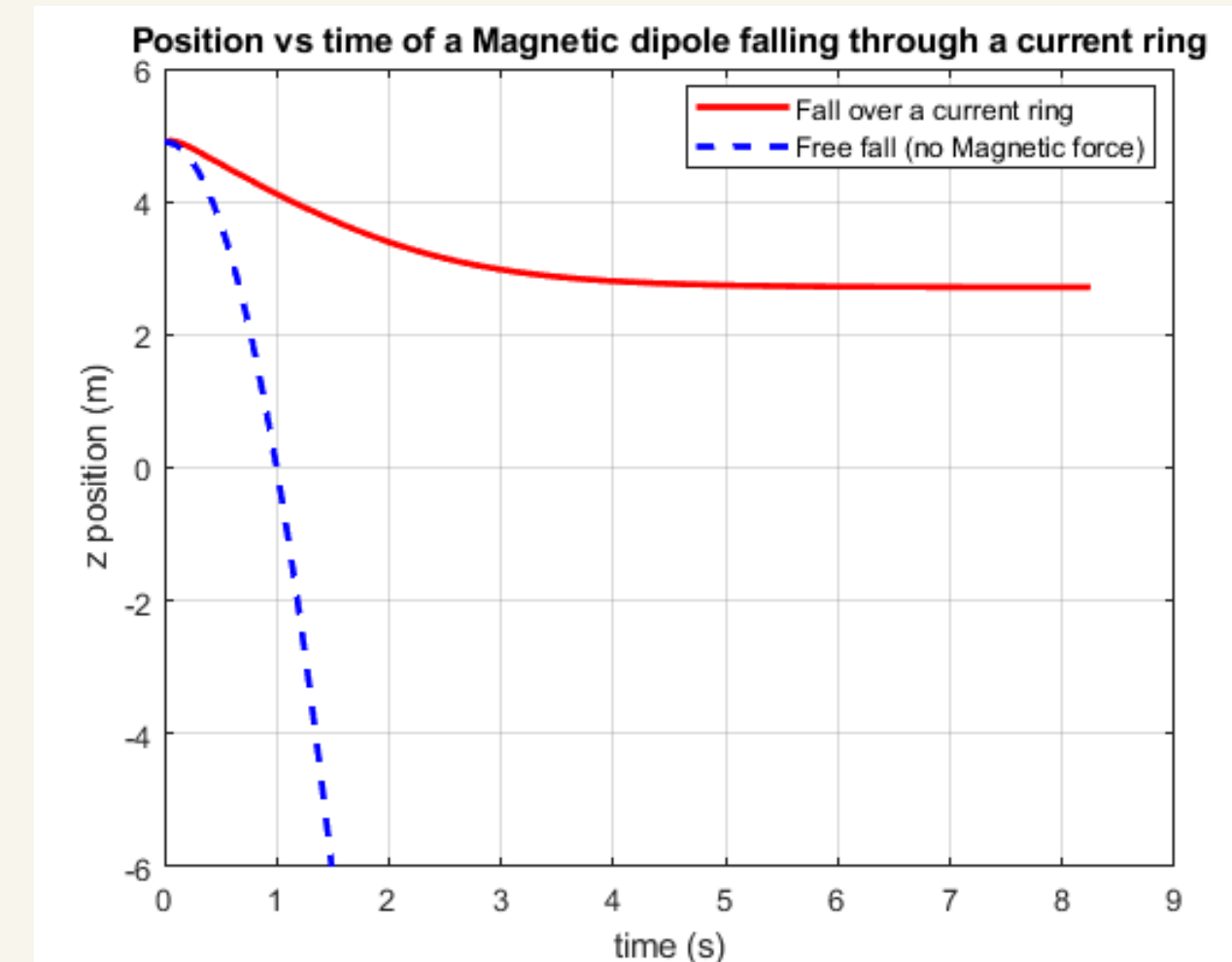
- ds: step size for the z-position vector, controls spatial resolution.
- km: proportional constant, related to $\mu_0 * I / (4\pi)$.
- Px, Py, Pz: vectors with the coordinates of the current elements.
- dx, dy: components of the current vector for each element.
- ne: number of elements per coil.
- N: number of coils.
- rw: regularization parameter to avoid division by zero in $\hat{r}$.
- plot_option: if true, generates a full flow-type field map; if false, generates only the axial field gradient profile.





TRAYECTORY

The trajectory shows the direction in which the magnet moves through the solenoid, compared to the path it would follow if it were not affected by a magnetic field. Using a while loop, both motion paths are plotted point by point, showing their position along the z-axis and displayed in the same graph—blue for the motion without a magnetic field, and red for the motion through the solenoid.



ACCELERATION FUNCTION

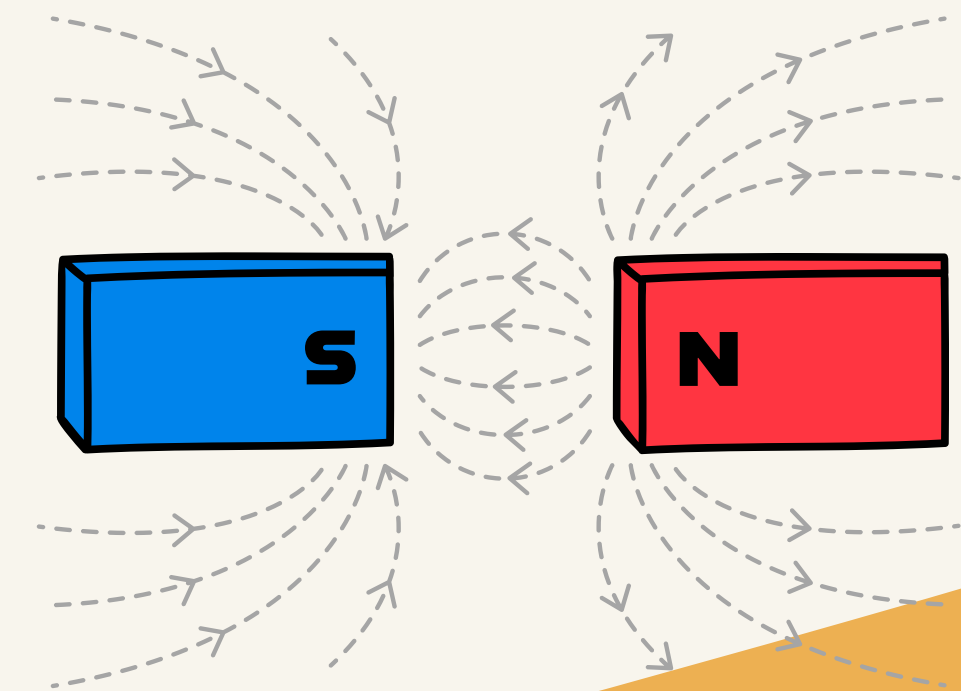
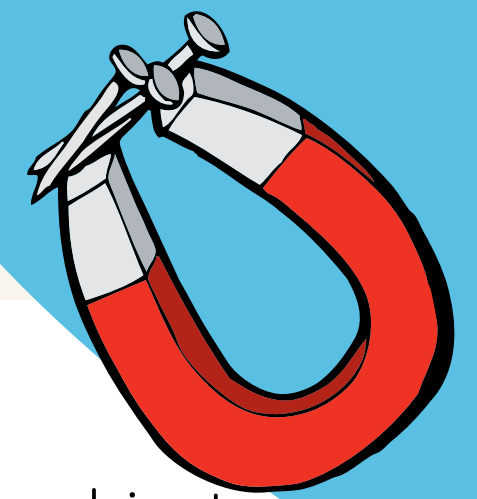
The acceleration function uses the magnetic force calculated from the magnetic field along the z-axis to determine the object's acceleration as it moves through the solenoid's magnetic field.

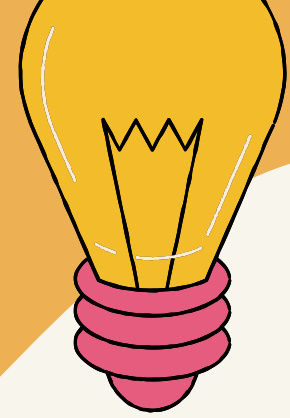
```
function a = a_total(z, v, Bz, z_axis, mag, gamma, m)
    delta = 0.005;
    Bz_forward = interp1(z_axis, Bz, z + delta, 'linear', 'extrap');
    Bz_backward = interp1(z_axis, Bz, z - delta, 'linear', 'extrap');
    dBz_dz = (Bz_forward - Bz_backward) / (2 * delta);

    % Calcula fuerza magnética y total
    Fm = -mag * dBz_dz;
    Ff = -gamma * v; % Nueva fuerza de fricción
    F = Fm + Ff - m * 9.81;
    a = F / m;
end
```

PARAMETERS

- z: The current z-position of the object.
- v: The current velocity of the object.
- Bz: The magnetic field along the z-axis.
- z_axis: The intervals where the function is applied along z.
- mag: The magnitude of the magnetic force.
- gamma: Coefficient used to calculate the frictional force.
- m: Mass of the object.





RUNGE KUTTA FUNCTION

The Runge-Kutta function implements the fourth-order Runge-Kutta method to calculate the position in z and the velocity. It follows the standard steps where k_1 is the initial slope, k_2 is the slope at the midpoint using the initial direction, k_3 is the midpoint slope using the correction from k_2 , and k_4 is the slope at the end of the step.

$$\begin{aligned} k_1 &= f(x_n, y_n) && \text{pendiente inicial} \\ k_2 &= f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_1\right) && \text{pendiente al punto medio usando la dirección inicial} \\ k_3 &= f\left(x_n + \frac{h}{2}, y_n + \frac{h}{2}k_2\right) && \text{pendiente al punto medio usando la corrección de } k_2 \\ k_4 &= f(x_n + h, y_n + h \cdot k_3) && \text{pendiente al final del paso} \end{aligned}$$
$$\rightarrow y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

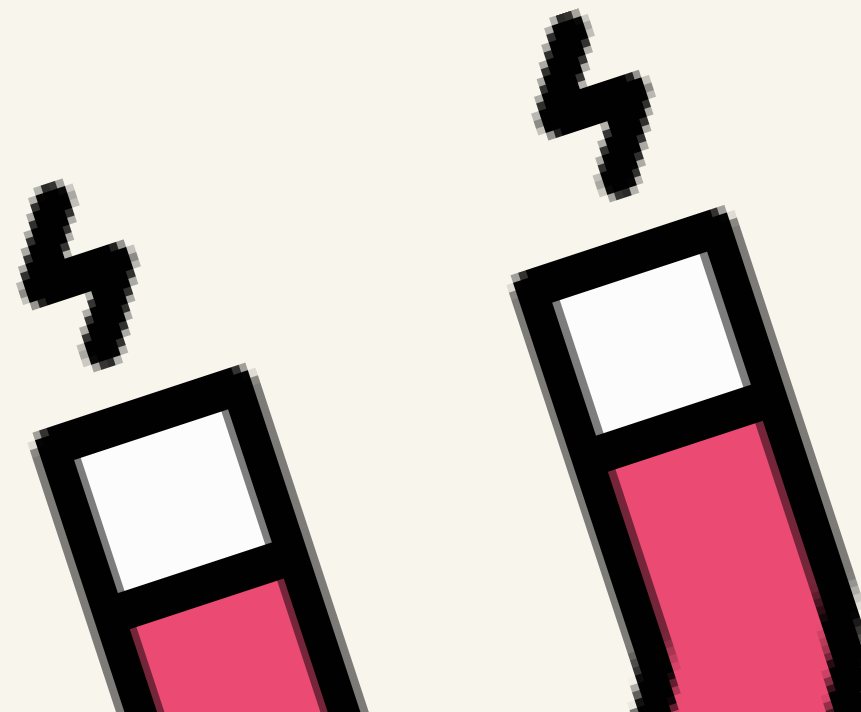
```
function [z_next, v_next] = metodoRungeKutta(z, v, dt, a_func)
    k1z = v;
    k1v = a_func(z, v);

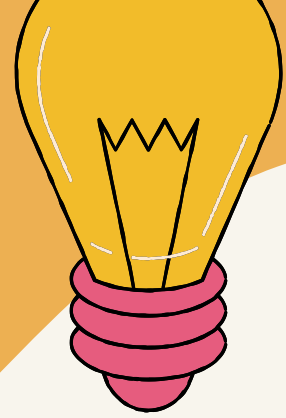
    k2z = v + 0.5*dt*k1v;
    k2v = a_func(z + 0.5*dt*k1z, v + 0.5*dt*k1v);

    k3z = v + 0.5*dt*k2v;
    k3v = a_func(z + 0.5*dt*k2z, v + 0.5*dt*k2v);

    k4z = v + dt*k3v;
    k4v = a_func(z + dt*k3z, v + dt*k3v);

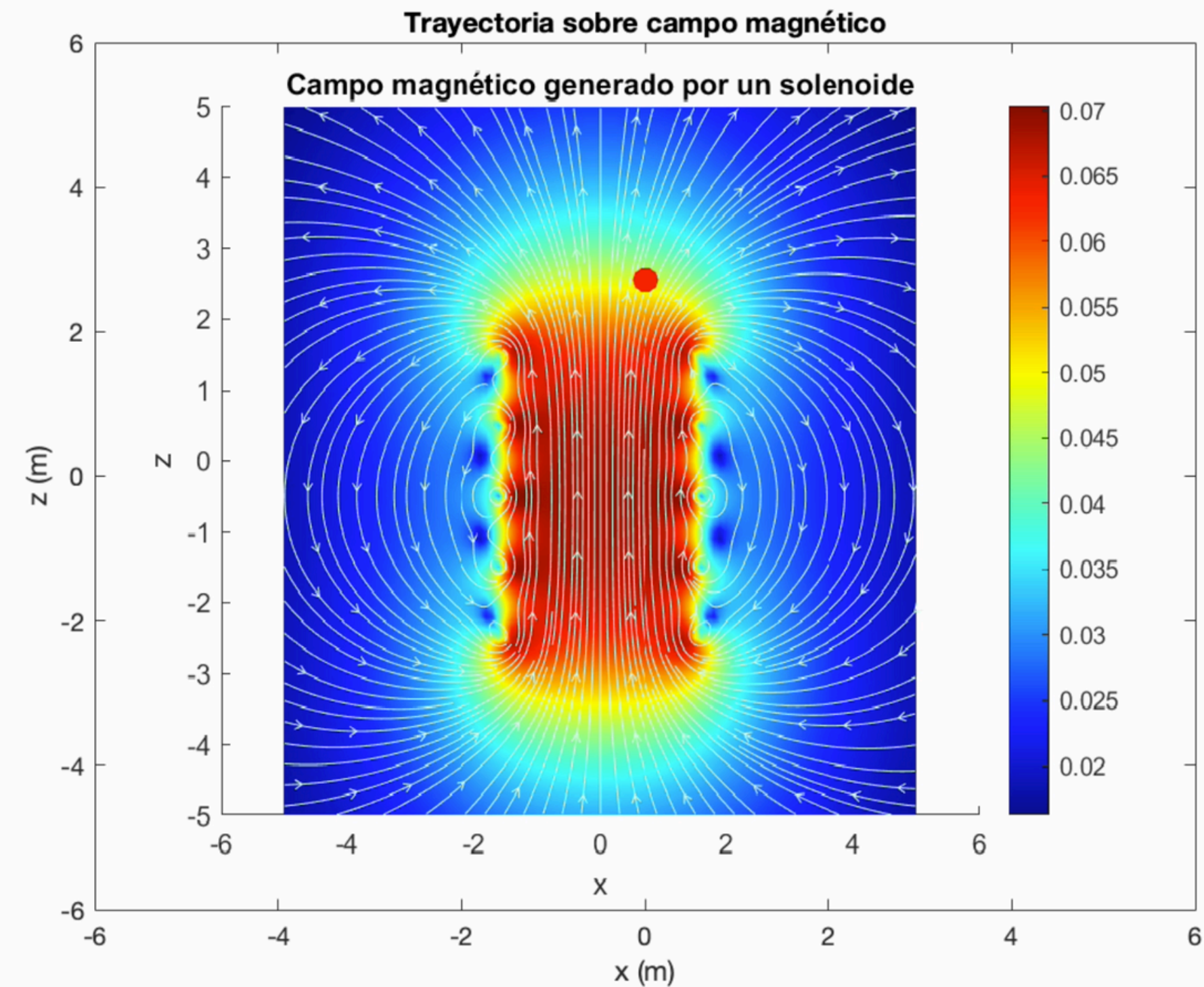
    z_next = z + (dt / 6) * (k1z + 2*k2z + 2*k3z + k4z);
    v_next = v + (dt / 6) * (k1v + 2*k2v + 2*k3v + k4v);
end
```



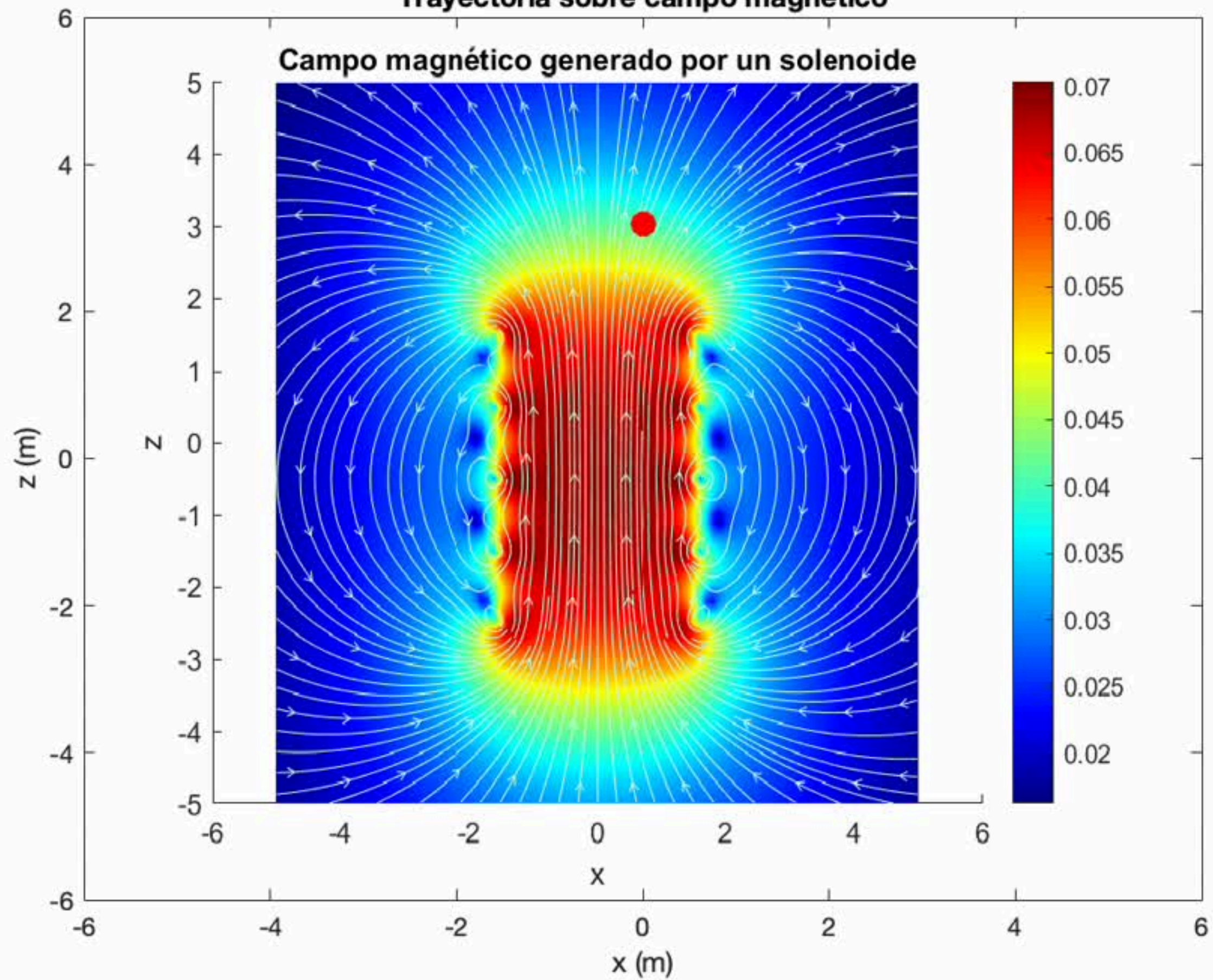


ANIMATION

In the animation, we can observe the magnetic field of the solenoid and a magnet that is dropped into it. The magnet begins to fall due to gravity, but as it enters the magnetic field generated by the solenoid, an opposing electromagnetic force is produced. We can see how its speed gradually decreases until it becomes suspended, reaching a state of equilibrium where the upward magnetic force balances its weight.



Trayectoria sobre campo magnético





i THANKS!